AI-driven SDLC: через призму системного мышления

Провокационный вопрос

- Кто использует AI в работе?
- А теперь - кто уверен, что использует AI правильно на всех этапах разработки?
- Не только в кодинге - а от идеи до продакшена?

Обо мне

- Полосов Юрий
- 20 лет в IT
- Solution Architect & Platform Engineer в iGaming Platform startup

> Сегодня я хочу разобраться вместе с вами: что значит "**правильно**" применять AI в SDLC - и почему для ответа на этот вопрос нужно **системное мышление**.

Цепочка создания ценности: выпечка пирожков

Но прежде чем говорить про AI и SDLC - давайте отойдем от кода и посмотрим на то, как вообще создается **ценность**. Возьмем понятный пример - **выпечка пирожков**.

> Пекарь печет пирожки и продает их прохожим. Пирожки он печет в промышленной духовке, которую купил на фабрике. Фабрика производит духовки на конвейере. А строительство фабрики и наладку конвейера заказали у компании, которая специализируется на промышленном пекарном оборудовании.

- Где здесь ПО?
- Всё ли здесь ПО?
- И где именно мы можем применить AI?

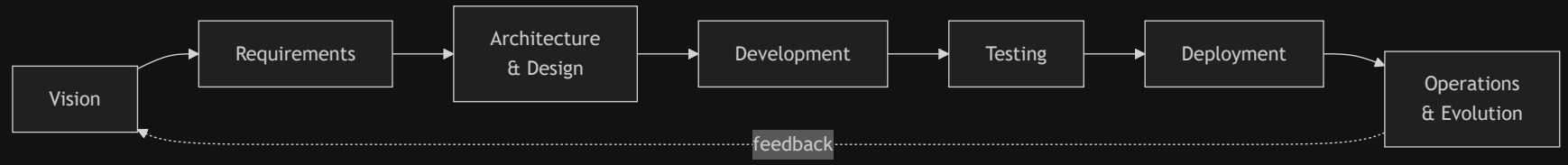
Цифровой двойник

- Ключевое понятие - **цифровой двойник**. Это ПО, повторяющее бизнес-процессы из реальной жизни.
- Важный нюанс: цифровой двойник часто охватывает не всю цепочку создания ценности - а только ту часть, которую бизнес решил цифровизировать.

Классический Software Development LifeCycle

- Хорошо, с цепочками ценности разобрались. А как мы обычно создаем ПО?

Канонический набор фаз



Популярные методологии, фреймворки, практики

- **Waterfall** - линейный, последовательный.
- **V-Model** - waterfall + параллельное тестирование.
- **Scrum** - спринты, роли, церемонии.
- **Kanban** - поток, WIP-лимиты, визуальная доска.
- **DevOps** - объединение Dev и Ops, CI/CD.

SDLC в цепочке создания ценности для пекарни

Теперь попробуем наложить SDLC на нашу цепочку пекарни. И тут возникает **проблема** - цепочка-то не одна!

Цепочек несколько!

- фабрика производит духовки
- духовка печет пирожки
- пирожок питает энергией прохожего

Вопросы к залу:

Так для какой из них мы создаем **цифрового двойника**?

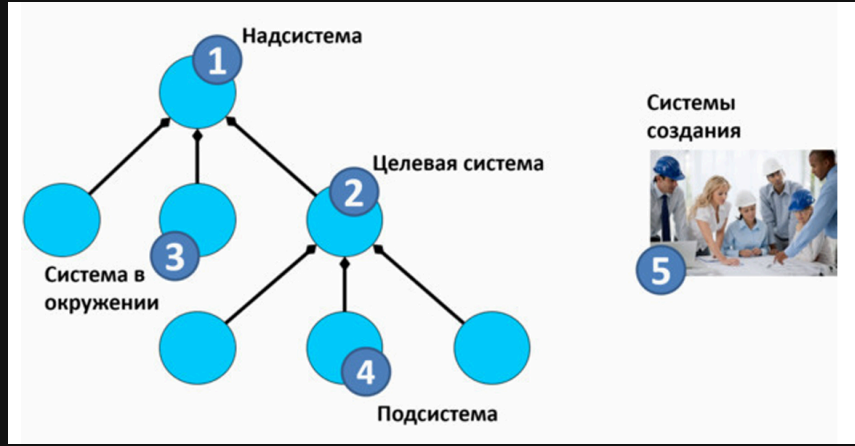
Где **целевая система**?

Виды систем

Чтобы ответить, нам нужен язык для разговора о системах - **системное мышление**.

Виды систем:

- **целевая система** - то, что мы создаем
- **подсистема** - часть целевой
- **надсистема** - контекст, в который встраивается целевая
- **система в окружении** - соседние системы на одном уровне с целевой в надсистеме
- **система создания** - то, чем мы создаем целевую



Владение системой

Представьте: компания-производитель духовок сама построила пекарню, сама печет и сама продает пирожки.

Кто теперь владеет системой - и что это меняет?

Примеры моделей владения

Dogfooding - владеем системой как тестировщики, находим проблемы до клиентов.

Reference Implementation - владеем системой как витриной, показываем клиентам качество.

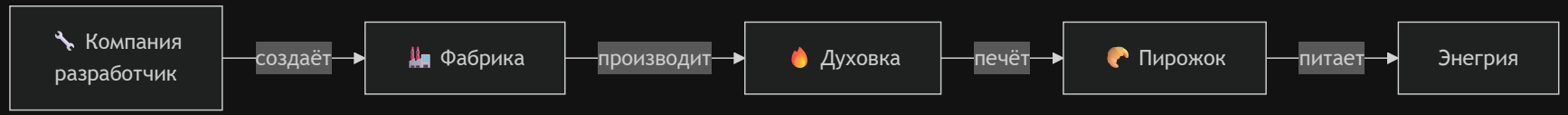
Captive Operator - владеем всей цепочкой, производим, печем и продаём сами.

> Когда владельцы разные - границы систем очевидны.

> Когда владелец один - границы размываются.

Выбираем целевую систему

Давйте выберем целевую систему.



Но к чему именно применяется SDLC?

- К целевой системе?
- А может, к подсистеме?
- К системе в окружении?
- К надсистеме?
- К системе создания?

Operational vs Development Value Stream

Оказывается, цепочки создания ценности бывают двух видов:

Operational Value Stream (OVS) - как мы используем систему

Development Value Stream (DVS) - как мы создаем систему

> И вот ключевой инсайт: SDLC можно рассматривать и как систему создания, и как целевую систему!

Вопрос залу:

- Было ли у вас такое - вы убеждаете заказчика, что изучение нового фреймворка улучшит его продукт?
- Фреймворк - это целевая система или система создания?
- Это OVS или DVS?
- А его изучение?
- А его использование?

SDLC как система создания

Итак, SDLC - и инструмент, и продукт одновременно.

Начнем с SDLC как инструмента.

Можем ли мы один SDLC применить ко всем целевым системам в цепочке (фабрика - духовка - пирожок)?

> Нет! Это разные системы с разными требованиями!

А чтобы понять требования - нужно сначала определить цель и масштаб проекта.

Фаза Vision

Цель и масштаб - этим занимается первый этап SDLC: **Vision**.

Но задумайтесь: у разработчика и бизнесмена одинаковые интересы?

Нет - они ортогональны!

Разные стейкхолдеры - разные concerns.

Кто, что и зачем делает в проекте

Здесь нам нужна точная терминология.

Пять важных понятий:

- предмет интереса - любая характеристика системы
- интерес (concern) - нацеленность на конкретное значение
- метод работы (навык) - дисциплинированный способ изменения состояния мира
- роль - функциональная позиция, определяемая методом работы. Роли играют.
- агент - конструктив, который исполняет роль. Человек, организация или AI.

Concern - Viewpoint - View

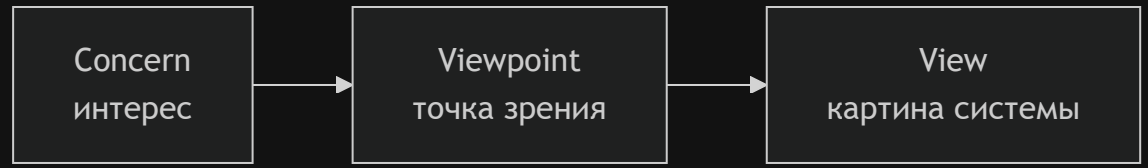
Разные предметы интереса

- Разработчик заинтересован в развитии системы создания (и себя в первую очередь!)
- Бизнесмен заинтересован в развитии целевой системы

Разные роли

- Вася в роли разработчика работает по методу кодирования на C#
- Петя в роли предпринимателя-визионера работает по методу целеполагания и стратегирования

Интересы (concerns) у них разные. Поэтому каждая роль смотрит на систему со своей точки зрения (viewpoint), а следовательно видит свою картину (view).

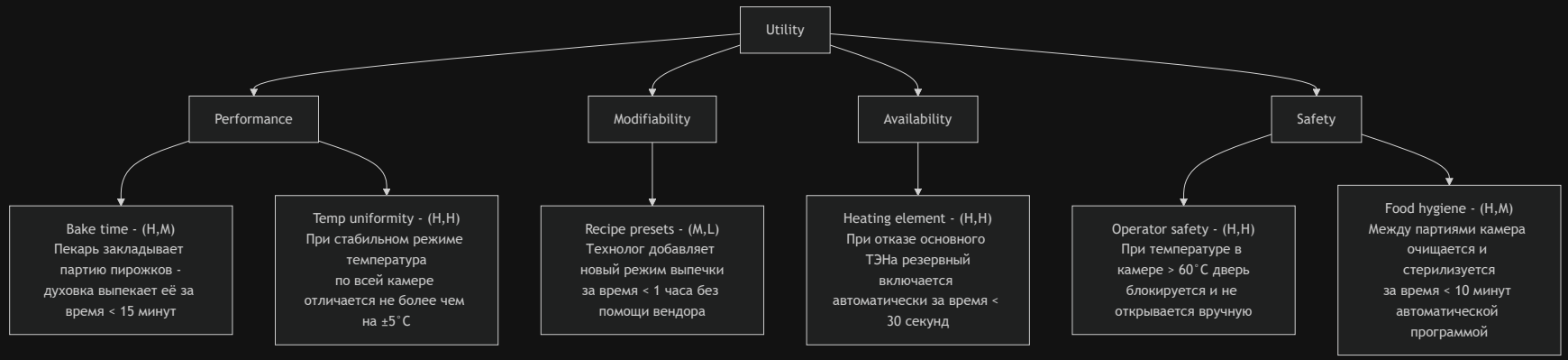


Разные точки зрения - разные картины системы. А значит - и разные атрибуты качества для каждой из них.

Quality attributes (QA)

Атрибуты качества - измеримые свойства системы.

Пример utility tree для целевой системы **промышленная духовка**:



Масштаб систем

Разные этапы SDLC так же имеют разные **concerns** (наборы атрибутов качества) - потому что на каждом этапе преобладают разные **роли**.

Но может ли один агент играть несколько ролей?

Конечно!

Вася делает **pet project** - он и визионер, и архитектор, и разработчик. Акцент зависит от того, какая роль преобладает в определенный момент.

Однако: у **команды агентов** (stakeholders) в разных ролях суммарно больше concerns, чем у одного агента во всех ролях сразу.

> Один агент во всех ролях - это **антипаттерн** "Бог".

Подход должен соответствовать масштабу системы:

- TOGAF для pet project - карго-культ
- Отсутствие CI/CD для enterprise - катастрофа

Ситуационная инженерия методов

Это и есть **Situational Method Engineering** - выбор подходов под масштаб.

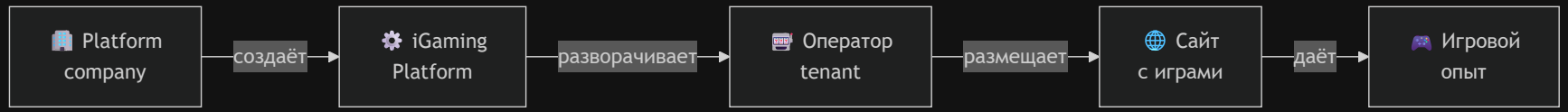
Phase	Pet project	Mid size	Enterprise solution
Vision	README + салфетка	Lean Canvas, VSM, стейкхолдер-анализ	BMC, ArchiMate, capability map, compliance
Requirements	TODO в голове	User Stories, AC, MoSCoW	Use Cases, SRS (IEEE 830), ISO 42010, трассировка
Architecture	«просто код»	ADR, C4 L1-2, ADD lite	ADD 3.0, ISO 42010 views, ATAM/QAW, PoC
Development	main-ветка, IDE	Git Flow, PR review, линтеры, CI	Feature toggles, inner source, CODEOWNERS
Testing	«покликал»	Unit + Integration, TDD, CI	Pyramid, contract, performance, security, chaos
Deployment	git push + Vercel	CI/CD, staging, blue-green	GitOps, canary, multi-region, rollback
Operations	логи в консоли	Grafana, alerting, SLO, runbooks	SRE, SLA/SLO/SLI, observability, post-mortems

Цепочка создания ценности: iGaming

Теперь применим все это к реальному кейсу из IT.

Вот цепочка создания ценности iGaming:

> Компания владеет и развивает платформу. Оператор разворачивает на платформе свой tenant, создает iGaming-сайт и продает доступ к играм пользователям. Оператор платит компании за эту возможность.



Владение системой: iGaming

А как бы выглядела эта цепочка с dogfooding, reference implementation и captive operator?

Компания-владелец платформы сама развернула тестовый tenant, создала тестовый iGaming-сайт с играми и продает доступ пользователям. Компания - сама себе клиент.

Примеры моделей владения

Dogfooding - владеем платформой как тестировщики, находим проблемы до операторов.

Reference Implementation - владеем tenant’ом как витриной, показываем операторам качество.

Captive Operator - владеем всей цепочкой, разрабатываем, разворачиваем и продаём сами.

Уровень 1: ЦС = Пирожок / Сайт с играми

Вид системы	Выпечка	iGaming
Целевая система	Пирожок	Сайт с играми
Надсистема	Едок (организм в момент еды)	Игрок (досуг, браузер)
Системы в окружении	Напиток, упаковка, салфетки	Social login, payment, support
Подсистемы	Тесто, начинка, корочка	Лобби, сессия, ставки
Система создания	Пекарь + духовка	Оператор + tenant

Уровень 2: ЦС = Духовка / Оператор tenant

Вид системы	Выпечка	iGaming
Целевая система	Духовка	Оператор tenant
Надсистема	Пекарня	Бизнес-операции оператора
Системы в окружении	Холодильник, миксер, касса	KYC, payment gateway, CRM
Подсистемы	ТЭНы, терморегулятор, камера	Tenant config, game catalog, DB
Система создания	Фабрика + конвейер	iGaming Platform

Уровень 3: ЦС = Фабрика / iGaming Platform

Вид системы	Выпечка	iGaming
Целевая система	Фабрика + конвейер	iGaming Platform
Надсистема	Индустрия пекарного оборудования	Рынок iGaming-операторов
Системы в окружении	Поставщики, логистика, регуляторы	Cloud, конкуренты, регуляторы
Подсистемы	Производственные линии, склад	Tenant provisioning, billing
Система создания	Компания-разработчик оборудования	Platform company

Выбираем целевую систему

Допустим, мы определили:

- Целевая система - iGaming site (B2C)
- Система создания - компания X
- Надсистема - игровой опыт пользователя (надсистема - это всегда то, что меняет целевая система)
- Атрибуты качества OVS:
  - Availability, Performance, Security, Scalability, Reliability
- Атрибуты качества DVS:
  - Deployability, Configurability, Traceability, Testability, Time-to-market
- Масштаб - условно Mid size

Вопрос залу:

- Почему Traceability - в DVS, а не в OVS?
- И почему Time-to-market - это именно DVS?

Прослеживаемость

Разберем Traceability подробнее.

Traceability - это податрибут более общего атрибута качества **Auditability** - способности в любой момент ответить: "почему мы приняли это решение?"

Почему это критично, когда агенты - люди?

> Потому что чем больше стейкхолдеров, тем важнее сохранять и передавать контекст на всех этапах.

Мемом vs феном

И тут нам поможет пара понятий из системного мышления:

Мемом - описание, замысел.

Феном - воплощение, реализация.

Мемом (описание)	→	Феном (воплощение)
Requirements / User Stories	→	**Реализованное поведение** системы
ADR (Architecture Decision Record)	→	**Архитектура работающей системы** (реализованные решения в коде и инфраструктуре)
Test spec	→	**Прошедший тест** (зелёный результат)
Код (исходный)	→	**Работающий продукт** (deployed, running system)

Промпт

А теперь добавим строку, которая все меняет:

Мемом (описание)	→	Феном (воплощение)
Промпт	→	**Сгенерированный артефакт** (код, тест, документ, конфигурация, **промпт**)

Промпт может быть мемомом и феномом.

А значит агент может создавать как мемомы так и феномы.

Мы ранее говорили, что разные viewpoints → разные роли.

Несколько ролей - это команда? Или один агент?

А агент - обязательно человек?

Что если AI-агент может играть некоторые из этих ролей?

Или их все?

Уровни автономности систем

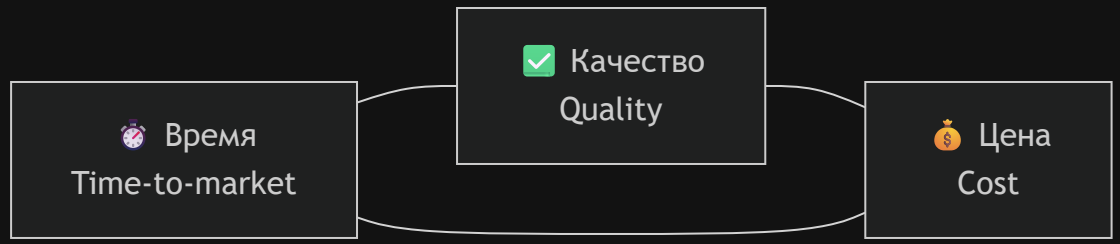
Три уровня автономности (Levels of Autonomy):

- **Human in the loop** - человек принимает каждое решение.
- **Human on the loop** - человек наблюдает и вмешивается при необходимости.
- **Human out of the loop** - система действует полностью автономно.

Последствия AI-driven SDLC

Зачем вообще внедрять AI в SDLC?

Ответ кроется в атрибутах качества - прежде всего **Time-to-market**.



Но если AI строит системы - то как его **проверять**?

Так же, как проверяем людей:

- детерминированные тесты, линтеры
- валидация и ревью другими агентами (людьми или AI)

Test-Driven Development

И здесь мы приходим к TDD.

Вопрос залу:

TDD - это про тестирование или про дизайн?

- **Unit-тест** - это мемом будущего фенома: реализованного **поведения**.
- **E2e-тест** - это мемом будущего фенома: реализованного **сценария использования**.
- **Fitness function** - это мемом будущего фенома: реализованной **архитектуры**

> Тесты - не проверка задним числом.
>
> Тесты - это описание желаемого будущего.
>
> А значит - идеальный **контракт** для AI-агента.

Цепочка создания ценности: AI-driven SDLC

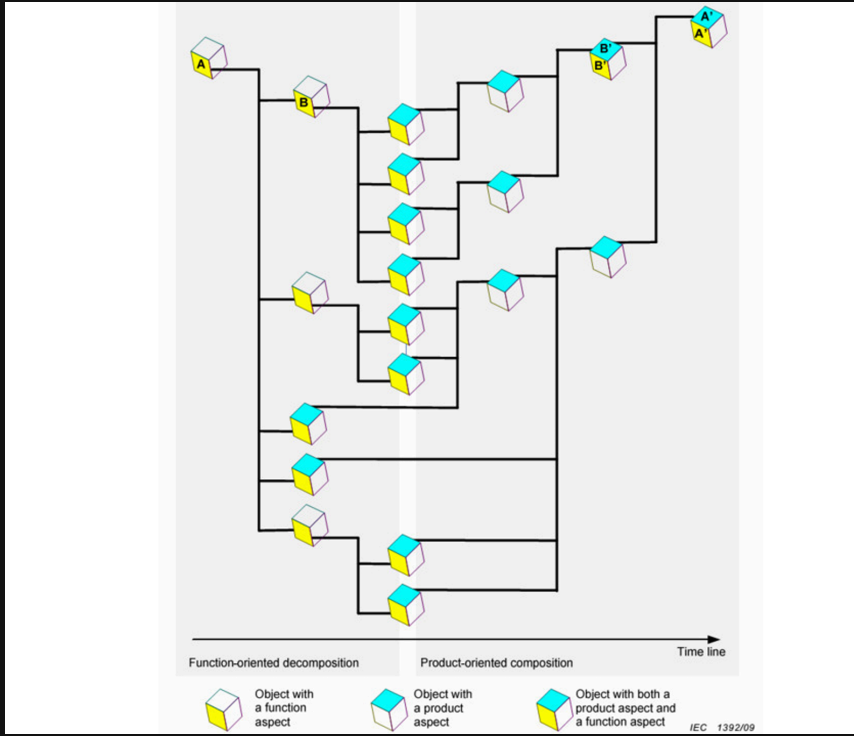
Рассмотрим ещё одну цепочку создания ценности:

> Компания разрабатывает iGaming Platform на заказ. Параллельно она создает open-source Claude Code plugin, который любой разработчик может использовать для создания софта.

Вопрос:

- А как понять, что внутри этого плагина?

Functional decomposition / Constructive composition



Функциональный анализ - рассмотрение системы через роли и методы работы.

Конструктивный синтез - подбор аффордансов-конструктивов для реализации ролей.

> Сначала определяем необходимые функции, потом подбираем реализующие их конструктивы.

Анатомия Claude Code plugin

Артифакты Claude Code	Артифакты Системного мышления
CLAUDE.md	Конституция проекта, документированные нормы культуры проекта
Rules	Локализованные нормы метода, привязанные к подсистемам
Skill	Метод работы, навык, мастерство (теория или алгоритм)
Subagent	Изолированный агент-исполнитель роли с preloaded методами
Command	Именованная точка входа в метод работы
Hook	Детерминированная fitness function, автоматический gate
MCP	Внешний конструктив-аффорданс, инструмент
Agent team	Оргструктура агентов в ролях с peer-to-peer координацией
Plugin	Система создания в сборе, инструментальный стек
Marketplace	Платформа распространения систем создания в сообществе

SDLC как целевая система

Итак, Claude Code plugin - это система создания.

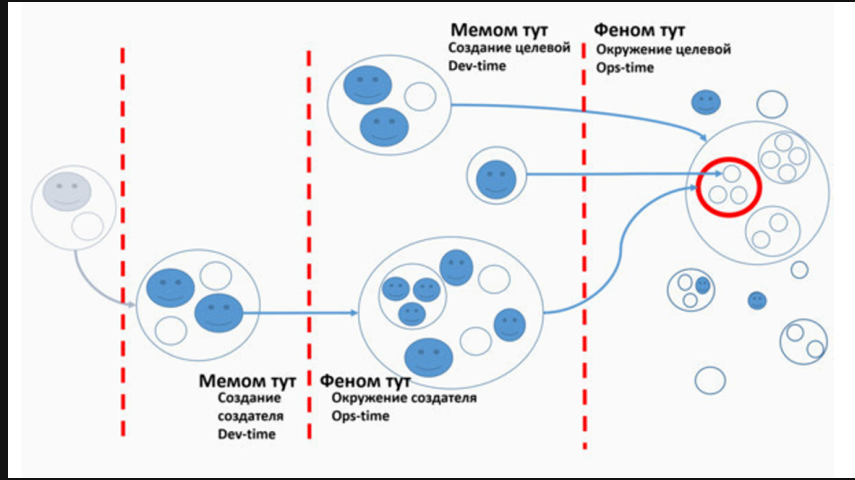
Но рано или поздно мы захотим развивать **сам плагин**.

Возможно, даже через тот же SDLC.

Получается, плагин становится **целевой системой**.

Вопрос к залу:

- А можем ли мы создавать его с помощью других плагинов?
- А с помощью самого себя?



Continuous everything

Это и есть Continuous Everything - непрерывность на всех горизонтах:

- **Dev-time** - непрерывная разработка
- **Ops-time** - непрерывная эксплуатация
- **Evo-time** - непрерывная эволюция

Сравнение аналогов

Что уже существует на рынке AI-driven development?

Инструмент	Слоган	Фокус
GSD (get-shit-done)	"The complexity is in the system, not in your workflow"	Context engineering, XML prompts
BMAD Method	"Build More Architect Dreams"	Agile + 12 personas, scale-adaptive
OpenSpec	"Agree before you build"	Lightweight specs, fluid workflow
spec-kit (GitHub)	"Specs become executable"	Constitution + spec → code
superpowers	"Agents work autonomously for hours"	TDD + systematic practices
AI-driven SDLC (наш)	"Через призму системного мышления"	Онтология создания систем

Чем мы отличаемся

Аналоги фокусируются на **workflow** - как организовать работу с AI для написания кода.

Мы фокусируемся на **онтологии** - как вообще думать о создании систем с участием AI.

Критерий	Аналоги	AI-driven SDLC
Фундамент	Workflow-практики	Системное мышление (Левенчук, ISO 42010, OMG Essence)
Онтология	Phases, personas, specs	Целевая система, роли, методы, мемом/феном
Применимость	Только к коду	К любым системам (трансдисциплинарно)
Масштаб	Quick vs full / scale-adaptive	Situational Method Engineering (Pet/Mid/Enterprise)
Continuous Everything	CI/CD	Dev-time + Ops-time + Evo-time
Self-application	-	Plugin создаёт сам себя

Demo: AI-driven SDLC в действии

А теперь - давайте соберем все вместе и увидим AI-driven SDLC в действии.

Демо

Применение Claude Code plugin как системы создания для простого TODO list приложения

Bonus Demo: AI-driven SDLC создает сам себя

И финальный твист: тот же подход я хочу применять к развитию самого плагина.

Демо

Claude Code plugin как система создания для самого себя как целевой системы

Спасибо за внимание!

Ресурсы

- Презентация: talk-ai-driven-sdlc
- Плагин: ai-driven-sdlc-plugin
- Анатолий Левенчук, "Системное мышление 2024" Том 1, Том 2
- Kazman/Cervantes, "Designing Software Architectures"
- code.claude.com/docs
- get-shit-done
- BMAD-METHOD
- OpenSpec
- spec-kit
- superpowers

Контакты

Полосов Юрий

Solution Architect & Platform Engineer

- Telegram: @ypolosov
- LinkedIn: ypolosov
- Email: ypolosov@gmail.com
- GitHub: ypolosov